Computational Sciences

Basic Git Commands

Luca Donati

M.Sc. Computational Sciences

Freie Universität Berlin

Winter semester 2024/2025

Contents

1 Gitlab of FU Berlin

Log in to the FU Berlin gitlab: https://git.imp.fu-berlin.de/users/sign_in

2 SSH key

An SSH key is a cryptographic key used for authentication between a client (your computer) and a server (such as GitLab, GitHub, or other remote servers) over the SSH (Secure Shell) protocol. SSH keys are an alternative to traditional username/password authentication, offering enhanced security. Instead of typing a password every time you connect to a server, SSH keys allow you to securely authenticate using a pair of cryptographic keys: a private key and a public key.

2.1 Generate a New SSH Key

1. Open your linux terminal.

2. Run the SSH key generation command:

   ```
   ssh-keygen -t ed25519 -C "your_email@example.com"
   ```

   where `your_email@example.com` is the email associated with your GitLab account. When prompted:

   - Press `Enter` to save the key in the default location (`~/.ssh/id_ed25519`).
   - Specify a different file name (press enter to skip this).

- Set a secure passphrase (press enter to skip this).

3. Add the SSH Key to Your SSH Agent with the command

   ```
   eval "$(ssh-agent -s)"
   ```

   Add your private key to the agent:

   ```
   ssh-add ~/.ssh/id_ed25519
   ```

   Replace `id_ed25519` with the correct file name if necessary.

4. Run the command

   ```
   cat ~/.ssh/id_ed25519.pub
   ```

   Copy the public key to your clipboard.

5. Log in to GitLab and navigate to:

   Profile Picture → Edit Profile → SSH Keys

   (a) Paste the copied public key into the provided text box.

   (b) Optionally, set an expiration date for the key.

   (c) Click **Add Key**.

3 Create a remote repository

1. Go to your gitlab page. Click on the + symbol at the top left and select 'New project/repository'.

2. Fill in the form fields and click on 'create project'.

4 Clone the remote repository locally

1. Open your linux terminal

2. Move to your working directory and create a directory named `local_repository`

```
mkdir local_repository
```

3. Move in the new directory:

```
cd local_repository
```

4. Initialize a Git repository:

```
git init
```

5. Clone the remote repository (the command is in one line):

```
git remote add origin https://git.imp.fu-berlin.de/username/NameRepository.git
```

To obtain the git address, click on 'code' in the main page of the repository and copy the address below 'Clone with HTTPS'.

6. Rename the main branch with

```
git branch -M main
```

7. Pull the content of the remote repository:

```
git pull origin main
```

With this command, you download the content of the remote repository into your local repository.

5 `git add`, `commit` and `push` files

The following commands are used to upload files and modifications from the local repository to the remote repository.

1. Create a new file in your local directory. For example use the program `vi` to create a new file and to edit it inside the prompt:

```
vi newfile.txt
```

Press the letter `i` to enter in the editor mode and write something. Press `Esc` to leave the editor mode. Write `:wq` and press `enter` to save

the file and close `vi`. Check that the file is in your directory with the command `ls`.

2. At this point, the directory in your local machine is different from the remote repository. Use the following commands to update the remote repository:

```
git add newfile.txt
git commit -m "Initial commit"
git push origin main
```

Difference between `git add`, `git commit` and `git push`

- `git add` prepares the changes, but does not save them permanently in the history.
- `git commit` saves changes in the repository history.
- `git push` upload the new files in the remote repository.

6 Branching

The repository is organized into branches. The main branch is called main; it is possible to create copies of the main branch and work separately on these copies, leaving the files of the main branch unchanged. This is useful, for instance, when working in groups or wanting to carry out tests without changing the code contained in the main branch.

1. Create a new branch:

```
git branch secondary
```

2. Switch to the new branch

```
git checkout secondary
```

3. Create a new file `newfile2.txt` using `vi`.

4. Add, commit and push the file on the new branch:

```
git add newfile2.txt
git commit -m "Add file on secondary branch"
git push origin secondary
```

You can check now on gitlab that two branches contain different files.